

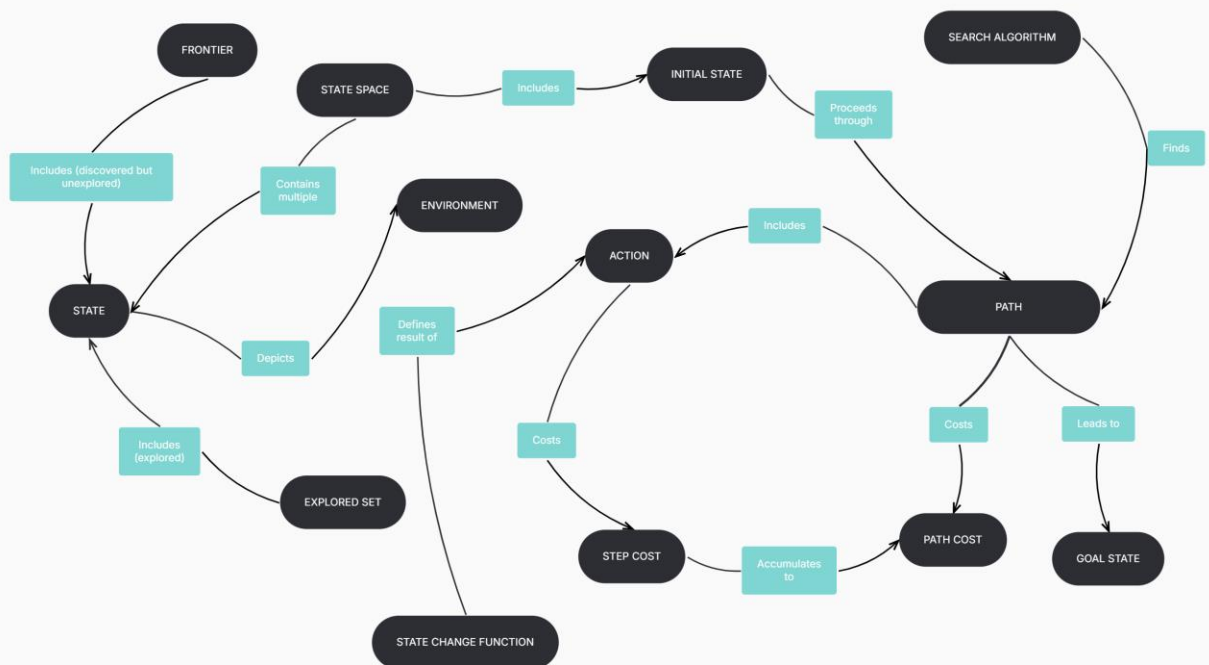
Applications of Artificial Intelligence

Unit 5 - Consolidation Task 1

Task 1 (Concept Mapping)

Your first task is to create a concept map that visually connects the following terms:

Environment, State, State Space, Initial State, Goal State, Action, Step Cost, Path Cost, State Change Function, Search Algorithm, Frontier, Explored Set.



(created on Mindmap: <https://app3.mindmap.com/map/?id=33da9697-0caa-4d64-b542-ce11a5e22651>)

Task 2 (Problem Formulation)

Task: Choose a real-world problem (e.g., finding the shortest route to a friend's house, organizing a study schedule, or solving a simple game such as tic-tac-toe). Write it in the formal structure.

Real-World Problem: Cooking Dinner

1. (S): **State space:** All possible combinations of ingredients and cooking processes.
2. (s): **Initial state:** With ingredients and cooking utensils, but without a cooked dinner.

3. (A): **Set of actions**: Various actions (chopping, heating, frying, steaming, sautéing, pouring)
4. $(f(s,a) \rightarrow s')$ **State change function**: $f(\text{Ingredients, Cook}) \rightarrow \text{Dinner}$
5. $(g(s) \rightarrow \text{bool})$: **Goal test function**: Checks if the present result is a solution (is this dish acceptable? Or is dinner done cooking?)
6. (c): **Action cost function**: The cost of each action taken (e.g. time taken to cook, cost of ingredients, electricity, etc).

Task 3: (Search Algorithm Analysis)

Search Algorithm Analysis

Breadth-first search explores the state space by each level.

- It is exhaustive, finding a solution where one exists.
- It finds the optimal path per node, locating the shortest path when all step costs are equal.
- By Big O notation, the time taken is $O(b^d)$, where b is branches by node, and d is depth of the shallowest goal.
- The space complexity is also $O(b^d)$, as all paths are kept in memory.
- Example: Searching for broken links in a website, where critical links are closer to the homepage, which is the starting point.

Depth-first search explores the state space by deep-diving along one path, then backtracking.

- It is systematic, but not exhaustive, as it initially concentrates on one path and only backtracks if this does not lead to a solution.
- It does not necessarily find an optimal solution because it can go infinitely deep down the wrong path.
- Time complexity is $O(b^m)$, where m is the maximum depth. DFS takes more time than BFS because m can be much larger than d , or even infinite.
- The space complexity (at $O(b \cdot m)$) is smaller than BFS, as memory stores only the current path.
- Example: Tree-structured file manager. The search dives a path, and backtracks when it reaches a dead-end, until the file is found.

A* search combines actual path cost, $g(n)$ with a heuristic estimate (informed guess) $h(n)$. This is represented as: $f(n) = g(n) + h(n)$.

- It is a complete search method.
- It is optimal, using an informed guess to decide which node to search first, saving time and cost. However, if the informed guess is too high, the search might skip over the optimal path, considering it too expensive.
- The time complexity is $O(b^d)$ at most, but a good heuristic estimate will trigger the search to stop long before this.
- The space complexity is also high ($O(b^d)$ as in BFS), as every node generated is stored in memory. A* search runs out of memory before running out of time on large problems.
- Example: Navigation software that applies a straight-line path as a heuristic, then develops a practical path, e.g., by adapting the heuristic to avoid roadblocks.